SORBONNE UNIVERSITÉ

Compte rendu de TP2

Par Ajinthini VAHEESAN

Etudiante en deuxième année de cycle ingénieur à Polytech Sorbonne spécialisé en électronique et informatique

Dans le cadre du cours de Système d'Exploitation

TRAVAIL PRÉSENTÉ À THIBAULT HILAIRE

MARS 2026

1 – Etape 1 : Des datagrammes avec accusé de réception

```
portable016@Portable-Polytech016: ~/EI4/S8/Système d'exp...
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ nano servudp.c
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ nano servudp.c
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ make clean
make
rm -f cliudp servudp
cc -Wall -o servudp servudp.c
cc -Wall -o cliudp cliudp.c
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./servudp
Le serveur est attache au port 9999 !
recu de 127.0.0.1 : <bonjour>
AR envoye a 127.0.0.1
```

```
portable016@Portable-Polytech016: ~/EI4/S8/Système d'exp...
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./cliudp 127.0.0.1 9999 bonjour
Envoi OK !
AR du serveur : <Bien reçu 5/5 !>
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$
```

Dans cette première étape, j'ai modifié les programmes UDP fournis afin d'ajouter un accusé de réception entre le client et le serveur.

Côté serveur, après la réception d'un message avec recvfrom(), j'ai ajouté un sendto() qui envoie la chaîne "Bien reçu 5/5 !" au client. L'option MSG_CONFIRM est utilisée, et l'adresse du client est récupérée directement depuis la structure sockaddr_in remplie par recvfrom(), ce qui permet de répondre automatiquement au bon expéditeur.

Côté client, après l'envoi du message avec sendto(), j'ai ajouté un recvfrom() pour récupérer la réponse du serveur, puis un printf() pour afficher l'accusé de réception.

Les tests montrent que le client reçoit bien le message "Bien reçu 5/5 !" et que le serveur affiche la réception puis l'envoi de l'AR, ce qui valide le bon fonctionnement de cette étape.

2 – Etape 2 : Programmes client/serveur pour l'identification

2.1 Mise en place du réseau pour les tests

```
portable016@Portable-Polytech016: ~/EI4/S8/Système d'exp...
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eno2: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN
group default qlen 1000
    link/ether 98:e7:43:7b:d0:19 brd ff:ff:ff:ff:ff:ff
    altname enp0s31f6
3: wlo1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group
 default qlen 1000
    link/ether 50:eb:71:e8:da:7d brd ff:ff:ff:ff:ff:ff
    altname wlp0s20f3
    inet 192.168.88.238/24 brd 192.168.88.255 scope global dynamic noprefixroute
 wlo1
       valid_lft 604748sec preferred_lft 604748sec
    inet6 fe80::1b39:c129:33a6:97bb/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$
```

Dans cette étape, j'ai configuré mon environnement réseau afin de pouvoir communiquer avec les autres machines du TP. Étant sous Linux, j'ai identifié mon interface wifi (wlo1) à l'aide de la commande ip a.

J'ai ensuite activé une interface secondaire avec ifconfig, puis je me suis connectée au réseau TPOSUSER via le gestionnaire wifi.

Enfin, j'ai vérifié la configuration avec ip a, ce qui m'a permis de confirmer que mon interface possède une adresse IP inet 192.168.88.238 Cette adresse correspond bien au réseau attendu, ce qui garantit le bon fonctionnement des communications en broadcast pour la suite du TP.

2.2 Création du serveur pour les tests

```
portable016@Portable-Polytech016: ~/EI4/S8/Système d'exp...
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ls
cliudp  cliudp.c  Makefile  servudp  servudp.c
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ nano servbeuip.c
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ nano Makefile
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ make clean
make
rm -f cliudp servudp servbeuip
cc -Wall -o servudp servudp.c
cc -Wall -o cliudp cliudp.c
cc -Wall -o servbeuip servbeuip.c
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ make clean
make trace
rm -f cliudp servudp servbeuip
cc -Wall -DTRACE -o servbeuip servbeuip.c
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./servbeuip Ajinthini
Serveur BEUIP actif sur le port 9998 avec le pseudo Ajinthini
Broadcast d'identification envoye
```

```
portable016@Portable-Polytech016: ~/EI4/S8/Système d'exp...
[TRACE] Datagramme recu depuis 192.168.88.238
[TRACE] Ajout couple : 192.168.88.238 - Ajinthini
[TRACE] Code recu : 1
[TRACE] Pseudo recu : Ajinthini
---- Table des presents ----
 1 : 192.168.88.238 - Ajinthini
----------------------------
[TRACE] AR envoye a 192.168.88.238
[TRACE] Datagramme recu depuis 192.168.88.238
[TRACE] Couple deja connu : 192.168.88.238 - Ajinthini
[TRACE] Code recu : 2
[TRACE] Pseudo recu : Ajinthini
---- Table des presents ----
 1 : 192.168.88.238 - Ajinthini
----------------------------
[TRACE] Datagramme recu depuis 192.168.88.189
[TRACE] Ajout couple : 192.168.88.189 - JUL
[TRACE] Code recu : 2
[TRACE] Pseudo recu : JUL
---- Table des presents ----
 1 : 192.168.88.238 - Ajinthini
 2 : 192.168.88.189 - JUL
----------------------------
```

Dans cette étape, j’ai développé le programme servbeuip.c à partir du serveur UDP avec accusé de réception réalisé précédemment. J’ai modifié le serveur pour qu’il utilise le port 9998 et prenne en paramètre un pseudo fourni par l’utilisateur.

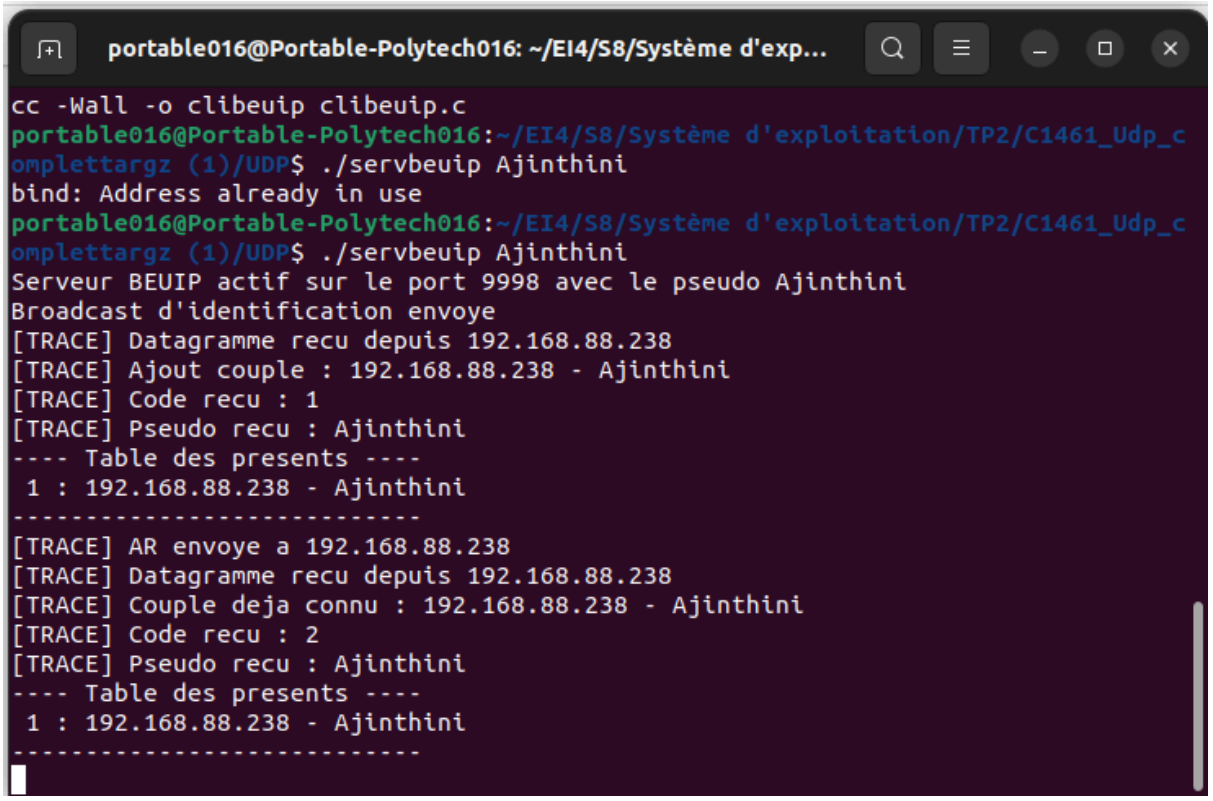
J’ai implémenté le format de message BEUIP avec un code ('1' pour le broadcast, '2' pour l’accusé de réception), la chaîne "BEUIP" pour identifier le protocole, puis le pseudo. Après le bind(), le serveur envoie automatiquement un message broadcast à l’adresse 192.168.88.255 contenant son pseudo.

Dans la boucle principale, le serveur reçoit les messages avec recvfrom(), vérifie leur validité (code et chaîne BEUIP), puis récupère l’adresse IP et le pseudo de l’expéditeur. Ces informations sont stockées dans une table de couples (IP + pseudo), en évitant les doublons.

Enfin, lorsque le serveur reçoit un message de code '1', il envoie un accusé de réception de code '2' contenant son propre pseudo. Les tests réalisés sur le réseau ont montré que le serveur récupère correctement les pseudos des autres machines et met à jour la table des présents.

2.3 Création du client pour les tests

Test 1 — lancer le serveur



Test 2 — afficher la liste des présents

Résultat côté client :

```
portable016@Portable-Polytech016: ~/EI4/S8/Système d'exp...
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./cliudp 127.0.0.1 9999 bonjour
Envoi OK !
AR du serveur : <Bien reçu 5/5 !>
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./clibeuip liste
Commande envoyee au serveur local
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$
```

Résultat côté serveur :

```
portable016@Portable-Polytech016: ~/EI4/S8/Système d'exp...
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./servbeuip Ajinthini
Serveur BEUIP actif sur le port 9998 avec le pseudo Ajinthini
Broadcast d'identification envoye
[TRACE] Datagramme recu depuis 192.168.88.238
[TRACE] Ajout couple : 192.168.88.238 - Ajinthini
[TRACE] Code recu : 1
[TRACE] Pseudo recu : Ajinthini
---- Table des presents ----
 1 : 192.168.88.238 - Ajinthini
----------------------------
[TRACE] AR envoye a 192.168.88.238
[TRACE] Datagramme recu depuis 192.168.88.238
[TRACE] Couple deja connu : 192.168.88.238 - Ajinthini
[TRACE] Code recu : 2
[TRACE] Pseudo recu : Ajinthini
---- Table des presents ----
 1 : 192.168.88.238 - Ajinthini
----------------------------
[TRACE] Datagramme recu depuis 127.0.0.1
---- Table des presents ----
 1 : 192.168.88.238 - Ajinthini
----------------------------
```

Test 3 — envoyer un message privé

```
portable016@Portable-Polytech016: ~/EI4/S8/Système d'exp...
----------------------------
[TRACE] Datagramme recu depuis 127.0.0.1
[TRACE] Datagramme recu depuis 192.168.88.236
[TRACE] Code recu : 1
[TRACE] Pseudo recu :
---- Table des presents ----
 1 : 192.168.88.238 - Ajinthini
----------------------------
[TRACE] AR envoye a 192.168.88.236
[TRACE] Datagramme recu depuis 192.168.88.230
[TRACE] Ajout couple : 192.168.88.230 - Geoffrey
[TRACE] Code recu : 1
[TRACE] Pseudo recu : Geoffrey
---- Table des presents ----
 1 : 192.168.88.238 - Ajinthini
 2 : 192.168.88.230 - Geoffrey
----------------------------
[TRACE] AR envoye a 192.168.88.230
[TRACE] Datagramme recu depuis 192.168.88.236
Message recu d'une IP inconnue :
[TRACE] Datagramme recu depuis 192.168.88.230
Message de Geoffrey : salut, ca marche ?
[TRACE] Datagramme recu depuis 127.0.0.1
```

```
portable016@Portable-Polytech016: ~/EI4/S8/Système d'exp...
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./cliudp 127.0.0.1 9999 bonjour
Envoi OK !
AR du serveur : <Bien reçu 5/5 !>
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./clibeuip liste
Commande envoyee au serveur local
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./clibeuip msg dede salut
Commande envoyee au serveur local
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./clibeuip msg dede salut
Commande envoyee au serveur local
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./clibeuip liste
Commande envoyee au serveur local
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./clibeuip all bonjour
Commande envoyee au serveur local
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./clibeuip msg Geoffrey salut
Commande envoyee au serveur local
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/C1461_Udp_c
omplettargz (1)/UDP$ ./clibeuip msg Geoffrey salut
```

Dans cette partie, j'ai développé un client clibeuip.c permettant d'envoyer des commandes au serveur BEUIP via des datagrammes UDP sur l'adresse locale 127.0.0.1:9998.

Le client permet d'exécuter plusieurs commandes. La commande liste (code '3') demande au serveur d'afficher la table des couples (adresse IP + pseudo). La commande msg (code '4') permet d'envoyer un message à un utilisateur spécifique. Le serveur recherche alors l'adresse IP correspondant au pseudo et envoie un message de code '9' contenant le texte. À la réception, le message est affiché sous la forme « Message de pseudo : message ».

La commande all (code '5') permet d'envoyer un message à tous les utilisateurs présents dans la table. Le serveur retransmet alors le message à toutes les adresses connues, sauf la sienne.

Une vérification a été ajoutée côté serveur afin que les commandes de type '3', '4' et '5' ne soient acceptées que si elles proviennent de l'adresse locale 127.0.0.1, ce qui permet d'éviter les utilisations malveillantes.

Les tests réalisés ont montré que toutes les fonctionnalités fonctionnent correctement : la liste des utilisateurs est affichée, les messages privés sont correctement transmis, et les messages globaux sont bien diffusés à tous les utilisateurs présents sur le réseau. Cela valide l'implémentation du protocole BEUIP.

3 – Etape 3 : Création des commandes internes

```
portable016@Portable-Polytech016$ ^C
portable016@Portable-Polytech016:~/EI4/S8/Système d'exploitation/TP2/biceps_v2$
./biceps
Commandes internes disponibles :
 - exit
 - cd
 - pwd
 - vers
 - beuip
 - mess
portable016@Portable-Polytech016$ beuip start Ajinthini
[TRACE] Nombre de mots : 3
Serveur BEUIP lance en fils pid=21534
portable016@Portable-Polytech016$ Serveur BEUIP actif sur le port 9998 avec le p
seudo Ajinthini
Broadcast d'identification envoye
[TRACE] Datagramme recu depuis 192.168.88.238
[TRACE] Ajout couple : 192.168.88.238 - Ajinthini
[TRACE] Code recu : 1
[TRACE] Pseudo recu : Ajinthini
---- Table des presents ----
 1 : 192.168.88.238 - Ajinthini
----------------------------
[TRACE] Datagramme recu depuis 192.168.88.238
[TRACE] Couple deja connu : 192.168.88.238 - Ajinthini
[TRACE] Code recu : 2
[TRACE] Pseudo recu : Ajinthini
---- Table des presents ----
 1 : 192.168.88.238 - Ajinthini
----------------------------
mess liste
[TRACE] Nombre de mots : 2
[TRACE] Datagramme recu depuis 127.0.0.1
---- Table des presents ----
 1 : 192.168.88.238 - Ajinthini
----------------------------
portable016@Portable-Polytech016$ mess all bonjour
[TRACE] Nombre de mots : 3
[TRACE] Datagramme recu depuis 127.0.0.1
portable016@Portable-Polytech016$ beuip stop
exit
[TRACE] Nombre de mots : 3
Serveur BEUIP arrete
portable016@Portable-Polytech016$
```

Dans cette dernière étape, j'ai restructuré le code développé précédemment en créant une librairie creme, chargée de regrouper les fonctionnalités réseau liées au protocole BEUIP. Pour cela, j'ai extrait du code utile des programmes servbeuip.c et clibeuip.c afin de créer un fichier creme.c, un fichier d'en-tête creme.h, ainsi qu'un Makefile adapté.

Les fonctions internes de la librairie ont été déclarées en static afin de ne pas être exportées inutilement, tandis que seules les fonctions nécessaires à biceps sont déclarées dans le fichier d'en-tête. La vérification avec nm et objdump permet ainsi de constater que seuls les symboles utiles sont visibles depuis l'extérieur.

J’ai ensuite intégré cette librairie dans une nouvelle version de biceps. La commande interne beuip start pseudo permet de lancer le serveur BEUIP dans un processus fils, tandis que la commande beuip stop lui envoie un signal pour l’arrêter proprement. Enfin, j’ai ajouté une commande interne mess permettant d’afficher la liste des utilisateurs présents sur le réseau, d’envoyer un message privé à un pseudo, ou d’envoyer un message à l’ensemble des utilisateurs.